

Experiment 8

Module Name: Unit Test Frameworks

Experiment Name: Data-Driven Automation (DDT)

Problem statement: Build a login script that reads multiple test cases from an external source (such as an Excel file via pandas or a local JSON/CSV file).

Objective: Build a data-driven testing framework that reads multiple test cases from a local CSV file, loops through the data, and dynamically executes the login script, validating the specific expected outcome for each row.

Tools, software, and concepts used: Python 3, Selenium WebDriver, the native csv module, Data-Driven Testing (DDT) methodology, Looping constructs.

Source code or implementation steps:

1. Create any Folder (Exa: Selenium)
2. Inside That Create two Folders 'test_data' & 'test'
3. Inside 'test_folder' create 'credentials.csv' and write this:

username	password	expected_error
standard_user	secret_sauce	None
locked_out_user	secret_sauce	Epic sadface: this user has been locked out.
invalid_user	wrong_pass	Epic sadface: Username and password do not match any user in this service

4. Now in the 'test' folder create 'test_login_ddt.py' and write this:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import csv
import time
import os

# Failsafe to always find the CSV file relative to where this script lives
current_dir = os.path.dirname(os.path.abspath(__file__))
csv_file_path = os.path.join(current_dir, "..", "test_data",
                              "credentials.csv")

def read_test_data(filepath):
    data = []
    with open(filepath, mode='r') as file:
        reader = csv.DictReader(file)
        for row in reader:
            data.append(row)
    return data

test_cases = read_test_data(csv_file_path)
```

```

for test in test_cases:
    driver = webdriver.Chrome()
    driver.maximize_window()
    try:
        driver.get("https://www.saucedemo.com/")

        # Inject dynamic data from the CSV
        driver.find_element(By.ID, "user-name").send_keys(test['username'])
        driver.find_element(By.ID, "password").send_keys(test['password'])
        driver.find_element(By.ID, "login-button").click()

        time.sleep(1)

        # Validation Logic based on expected outcome
        if test['expected_error'] == "None":
            assert "inventory.html" in driver.current_url
            print(f"PASS: Valid login worked for {test['username']}")
        else:
            error_msg = driver.find_element(By.CSS_SELECTOR, "h3[data-
test='error']").text

            # .strip() removes hidden spaces, and 'in' checks for a partial
            match
            assert test['expected_error'].strip() in error_msg.strip(),
f"Mismatch! Website said: '{error_msg}', but CSV expected:
'{test['expected_error']}'"

            print(f"PASS: Correct error caught for {test['username']}")

    finally:
        driver.quit()

```

5. Now in your created folder go to cmd and write this:

```
>>> python tests/test_login_ddt.py
```

Output with clearly labelled screenshots:

	A	B	C	D	E	F	G	H	I	J
1	username	password	expected_error							
2	standard_user	secret_sauce	None							
3	locked_out_user	secret_sauce	Epic sadface: this user has been locked out.							
4	invalid_username	wrong_password	Epic sadface: Username and password do not match any user in this service							
5										

Screenshot 1: A view of the raw credentials.csv file containing the test matrix

```
D:\My Desktop\Selenium>python tests/test_login_ddt.py
PASS: Valid login worked for standard_user
PASS: Correct error caught for locked_out_user
PASS: Correct error caught for invalid_user
```

Screenshot 2: Terminal output logging the PASS status for each of the 3 iterations dynamically

Brief Result: A single Python script successfully tested three entirely different login scenarios by iterating over external CSV data.

Observation: The script uses a conditional if/else block during validation to determine whether it should assert a successful URL redirect or assert the presence of an error message based on the CSV dataset.

Conclusion: Separating test data from execution logic (DDT) allows QA teams to rapidly scale test coverage. Non-technical users can add new test cases simply by adding rows to an Excel or CSV file without altering the underlying code.